Universidad del Valle de Guatemala Programación Orientada a Objetos
Segundo Ciclo, 2026
Catedrático: Kevin Antonio Velasquez Aguilar



Lab 3 - Herencia

Obed Natanael Tot Paiz
Carné:26971

Fecha de entrega: 18/10/2026

1. Análisis del problema

1.1. Requisitos funcionales

El sistema debe gestionar las órdenes de servicio de TecniHogar, organizar el trabajo de las cuadrillas y controlar los ingresos y la información se conservará en memoria mientras el programa se esté ejecutando.

1.1.1. Información

Elemento	**Información necesaria**
Orden de servicio	Código único, cliente, dirección, horas estimadas, zona, categoría, estado y visitas realizadas.
Instalación	Cantidad de puntos por instalar, además de la información común.
Instalación eléctrica certificada	Valor declarado de los materiales, además de los datos de instalación.
Mantenimiento de aire acondicionado	Capacidad del equipo en BTU, además de la información común.
Cuadrilla	Código único, capacidad máxima de horas por jornada y certificación en refrigeración.
Jornada	Cuadrilla responsable, órdenes asignadas, estado y resultados de las visitas.
Movimiento de ingreso	Orden asociada, concepto del cobro y monto.

Los totales de ingresos, las horas asignadas y la capacidad disponible se calcularán a partir de estos datos, para poder evitar guardar valores duplicados que puedan quedar desactualizados.

1.1.2. Operaciones y las condiciones de validez

Operación	**Condiciones de validez**

Cotizar	Mostrar costo y días hábiles de atención. Debe validar los datos del servicio, pero no registrar una orden ni generar ingresos.
Registrar una orden	Guardarla pendiente, con cero visitas, y registrar el cobro completo. El código debe ser único y no vacío.
Registrar una cuadrilla	Guardar su código, capacidad y certificación. Su código no puede estar vacío ni repetido y su capacidad debe ser positiva.
Crear una jornada	Asociarla con una cuadrilla disponible. Una cuadrilla no puede tener más de una jornada sin cerrar.
Asignar una orden	Requiere una jornada en preparación y una orden pendiente. No debe superar la capacidad, incumplir la certificación requerida ni pertenecer a otra jornada activa.
Retirar una orden	Solo durante la preparación y si pertenece a la jornada. La orden vuelve a pendiente; no genera visitas ni cobros.
Iniciar una jornada	Requiere que esté en preparación y tenga órdenes. Estas pasan a atención y aumenta su contador de visitas. Desde la segunda visita se cobran Q40 por orden.
Registrar un resultado	Requiere una jornada en curso y una orden en atención dentro de ella. Solo se admite un resultado por orden en esa jornada.
Cerrar una jornada	Ocurre automáticamente cuando todas sus órdenes tienen resultado. La cuadrilla queda disponible.
Consultar información	Mostrar las órdenes y las cuadrillas registradas.
Generar reportes	Mostrar órdenes por categoría y estado; ingresos por concepto y categoría; y jornadas abiertas con sus horas asignadas y capacidad disponible.

1.1.3.Reglas de costo y atención

Se define una tarifa base de Q100 para zona metropolitana y Q175 para zona periférica.

Servicio	Cálculo del costo	Tiempo de atención
Diagnóstico	Q150 en metropolitana; Q225 en periférica.	1 o 2 días hábiles, respectivamente.
Instalación	Tarifa base + Q90 × redondeo hacia arriba del mayor valor entre horas estimadas y puntos × 0.75.	3 días en metropolitana; 5 en periférica.
Instalación eléctrica certificada	Costo de instalación + Q120 + 5 % del valor declarado de materiales.	Dos días más que una instalación normal.
Mantenimiento de aire acondicionado	Tarifa base + Q110 × horas estimadas + Q80. Solo se admite zona metropolitana, por lo que su base efectiva es Q100.	2 días hábiles.

1.1.4. Validaciones y reglas generales

- Las horas estimadas y la capacidad de una cuadrilla deben ser positivas.
- Los datos iniciales deben incluir una instalacion cuyas horas por puntos superen sus horas estimadas.
- Un diagnóstico no puede superar 1.5 horas.
- Los puntos de instalación deben ser un entero positivo.
- El valor de materiales debe ser positivo, conforme a la restricción general de valores numéricos del enunciado.
- La capacidad de un aire acondicionado debe estar entre 9,000 y 60,000 BTU, inclusive.
- El mantenimiento de aire acondicionado solo se ofrece en zona metropolitana y requiere una cuadrilla certificada en refrigeración.
- Para la capacidad de la jornada se utilizan las horas estimadas, sin redondearlas ni sustituirlas por las facturables.
- Diagnósticos e instalaciones permiten hasta tres visitas, instalaciones certificadas, dos; mantenimientos, una.
- Una visita fallida devuelve la orden a pendiente si quedan intentos; si no quedan, la cancela. Una visita exitosa la completa, incluso si era el último intento permitido.
- Cotizar, asignar, retirar y rechazar operaciones no modifica los ingresos.
- Una operación inválida debe mostrar un mensaje y dejar intacta la información anterior.
- El menú debe permanecer disponible hasta salir; las entradas incorrectas no deben terminar el programa.
- Los montos se muestran con dos decimales.
- El programa inicia con dos órdenes pendientes por categoría como mínimo y tres cuadrillas, incluyendo una certificada y una no certificada. La carga inicial no genera ingresos.

1.2. Decisiones que dependen de la categoría

Las reglas particulares deben estar en las clases de servicio correspondientes.

Decisión	Clase responsable	Justificación
Precio fijo del diagnóstico	Diagnostico	Es una regla exclusiva de esta categoría.
Límite de 1.5 horas	Diagnostico	Determina qué diagnósticos son válidos.
Horas por puntos y redondeo de facturación	Instalacion	Forman parte de su cálculo de costo.
Validación de puntos positivos	Instalacion	También se aplica a las instalaciones certificadas.
Certificado y garantía de materiales	InstalacionElectricaCertificada	Amplía el costo de una instalación.
Validación del valor de materiales	InstalacionElectricaCertificada	Ese dato solo existe en esta categoría.
Costo por horas y gas refrigerante	MantenimientoAireAcondicionado	Es su regla particular de cobro.
Rango de BTU y zona permitida	MantenimientoAireAcondicionado	Son restricciones propias del mantenimiento.
Compatibilidad con una cuadrilla	esCompatibleCon()	Permite que cada categoría defina sus requisitos.
Días de atención	calcularDiasAtencion()	El plazo cambia según categoría y zona
Máximo de visitas		La política depende del tipo de servicio.

Jornada pregunta a la orden si es compatible con la cuadrilla; no necesita identificar su tipo concreto. De la misma manera, el sistema solicita el costo y el máximo de visitas mediante métodos polimórficos. El cargo de Q40 es común a todas las categorías. Se registra desde SistemaTecniHogar al iniciar la jornada.

1.3. Estados de las órdenes y jornadas

1. 3.1. Estados de una orden

Se propone un estado explícito ASIGNADA para distinguir las órdenes pendientes disponibles de las que ya están reservadas para una jornada.

Estado inicial	Operación o condición	Estado final
Sin registrar	Registrar la orden	PENDIENTE
PENDIENTE	Asignar a una jornada en preparación	ASIGNADA
ASIGNADA	Retirar de la jornada	PENDIENTE
ASIGNADA	Iniciar la jornada	EN ATENCIÓN
EN ATENCIÓN	Registrar resultado exitoso	COMPLETADA
EN ATENCIÓN	Registrar visita fallida y quedan intentos	PENDIENTE
EN ATENCIÓN	Registrar visita fallida y se agotaron los intentos	CANCELADA

Las órdenes completadas y canceladas no pueden asignarse nuevamente, el contador de visitas aumenta al iniciar la jornada, no al asignar la orden.

1. 3.2. Estados de una jornada

Estado inicial	Operación o condición	Estado final
Sin crear	Crear una jornada	EN_PREPARACION
EN_PREPARACION	Asignar o retirar órdenes	EN_PREPARACION
EN_PREPARACION	Iniciar con al menos una orden	EN_CURSO
EN_CURSO	Registrar un resultado y quedan otros pendientes	EN_CURSO
EN_CURSO	Registrar el último resultado pendiente	CERRADA

Una jornada en curso ya no permite asignar ni retirar órdenes. Una jornada cerrada conserva sus datos para consulta.

1.4. Clases, responsabilidades, atributos y métodos

En las tablas, - significa privado, + público, # protegido y ~ acceso de paquete.

1.4.1. OrdenServicio — clase abstracta

Representa los datos y el ciclo de vida comunes de las órdenes.

Atributo	Tipo	Propósito
- codigo	String	Identificar la orden.
- cliente	String	Guardar el nombre del cliente.
- direccion	String	Ubicar el servicio.
- horasEstimadas	double	Representar el trabajo previsto.
- zona	Zona	Indicar metropolitana o periférica.
- estado	EstadoOrden	Controlar las operaciones permitidas.
- visitasRealizadas	int	Contar los intentos iniciados.

Método	Resultado y propósito
# OrdenServicio(String codigo, String cliente, String direccion, double horasEstimadas, Zona zona)	Valida los datos comunes e inicializa el estado pendiente y las visitas en cero.
+ calcularCosto()	double, abstracto. Devuelve el precio del servicio.
+ calcularDiasAtencion()	int, abstracto. Devuelve los días hábiles estimados
+ getMaximoVisitas()	int, abstracto. Devuelve el límite de intentos.
+ esCompatibleCon(Cuadrilla cuadrilla)	boolean, abstracto. Determina si la cuadrilla

	puede atenderla.
+ getCategoria()	String, abstracto. Identifica la categoría para consultas y reportes.
# calcularTarifaBase()	double. Devuelve Q100 o Q175 según zona.
~ marcarAsignada()	void. Cambia de pendiente a asignada.
~ retirarAsignacion()	void. Devuelve una orden asignada a pendiente.
~ iniciarVisita()	void. Cambia a atención y aumenta las visitas.
~ registrarResultado(ResultadoVisita resultado)	void. Completa, devuelve a pendiente o cancela según el resultado y los intentos.

Los atributos tienen métodos públicos de consulta, con el tipo correspondiente, como getCodigo(): String y getEstado(): EstadoOrden.

1.4.2. Clases concretas de servicio

Clase	**Atributos propios**	**Constructor y comportamiento**
Diagnostico	Ninguno	Constructor público con los datos comunes. Valida el límite de horas. Calcula precio fijo, plazo según zona, tres visitas y compatibilidad con cualquier cuadrilla.
Instalacion	- puntos: int	Constructor público con los datos comunes y int puntos, valida puntos positivos y calcula costo por horas facturables, plazo según zona y tres visitas; admite cualquier cuadrilla.
InstalacionElectricaCertificada	- valorMateriales: double	Constructor público con los datos comunes, int puntos y double valorMateriales. Añade certificado y garantía, suma dos días al plazo y limita a dos visitas.

		Hereda la compatibilidad de instalación.
MantenimientoAireAcondicionado	- btu: int	Constructor público con los datos comunes y int btu. Valida zona y capacidad. Calcula costo por horas y gas, plazo de dos días y una visita. Exige certificación en refrigeración.

Método	**Propósito**
# calcularHorasFacturables(): double	Obtiene el mayor valor entre horas estimadas y puntos × 0.75, y lo redondea hacia arriba.

Los datos particulares se consultan mediante getPuntos(): int, getValorMateriales(): double y getBtu(): int, todos públicos.

1.4.3. Cuadrilla

Representa un equipo y sus capacidades

Atributo	**Tipo**	**Propósito**
- codigo	String	Identificar la cuadrilla.
- capacidadHoras	double	Limitar las horas por jornada.
- certificadaRefrigeracion	boolean	Indicar si puede atender mantenimientos de aire acondicionado.

Su constructor público es Cuadrilla(String codigo, double capacidadHoras, boolean certificadaRefrigeracion). Valida código no vacío y capacidad positiva.

Sus métodos públicos son getCodigo(): String, getCapacidadHoras(): double y isCertificadaRefrigeracion(): boolean. La disponibilidad se determina consultando las jornadas abiertas; no se guarda otro indicador que pueda contradecirlas.

1.4.4. Jornada

Organiza las órdenes de una cuadrilla y controla la ejecución de sus visitas.

Atributo	Tipo	Propósito
- id	int	Identificar la jornada mediante un número generado por el sistema.
- cuadrilla	Cuadrilla	Mantener el equipo responsable.
- ordenes	List<OrdenServicio>	Conservar las órdenes asignadas.
- resultados	Map<String, ResultadoVisita>	Guardar el resultado de cada orden en esta jornada.
- estado	EstadoJornada	Controlar la etapa de la jornada.

Método	Resultado y propósito
+ Jornada(int id, Cuadrilla cuadrilla)	Inicializa colecciones vacías y estado en preparación.
~ asignarOrden(OrdenServicio orden)	void. Valida estado, capacidad y compatibilidad antes de asignar.
~ retirarOrden(OrdenServicio orden)	void. Retira durante la preparación y deja la orden pendiente.
~ validarInicio()	void. Verifica que la jornada este en preparacion, tenga ordenes y que todas puedan iniciar su visita.
~ iniciar()	void. Inicia la jornada y las visitas de sus órdenes.
~ registrarResultado(String codigoOrden, ResultadoVisita resultado)	void. Registra el resultado, actualiza la orden y cierra la jornada cuando corresponde.
+ calcularHorasAsignadas()	double. Suma las horas estimadas.

+ calcularCapacidadDisponible()	double. Resta las horas asignadas a la capacidad.
+ getId(), + getCuadrilla(), + getEstado()	Devuelven int, Cuadrilla y EstadoJornada, respectivamente.
+ getOrdenes()	List<OrdenServicio>. Devuelve una vista de consulta que no permite modificar la colección.

1.4.5. Movimiento Ingreso
Representa un cobro realizado.

Atributo	**Tipo**	**Propósito**
- orden	OrdenServicio	Asociar el cobro con una orden y su categoría.
- concepto	ConceptoIngreso	Distinguir registro y visita adicional.
- monto	double	Guardar el valor efectivamente cobrado.

Su constructor público recibe esos tres valores. Ofrece getOrden(), getConcepto() y getMonto(), públicos y con los tipos correspondientes, sus datos no cambian después de crearlo.

1.4.6. SistemaTecniHogar

Coordina las operaciones y mantiene los registros generales.

Atributo Privado	**Tipo**	**Propósito**
ordenes	Map<String, OrdenServicio>	Guardar y buscar órdenes por código.
cuadrillas	Map<String, Cuadrilla>	Guardar y buscar cuadrillas por código.
jornadas	Map<Integer, Jornada>	Guardar y buscar jornadas por identificador.

ingresos	List<MovimientoIngreso>	Conservar los cobros realizados.
siguienteIdJornada	int	Generar el identificador de cada jornada.

Todos los métodos son publicos

Método y resultado	Propósito
SistemaTecniHogar()	Inicializa colecciones, contador y datos de ejemplo.
registrarOrden(OrdenServicio orden): void	Valida código único, guarda y cobra la orden.
registrarCuadrilla(Cuadrilla cuadrilla): void	Valida código único y guarda la cuadrilla.
crearJornada(String codigoCuadrilla): Jornada	Crea una jornada si la cuadrilla está disponible.
asignarOrden(int idJornada, String codigoOrden): void	Coordina la asignación.
retirarOrden(int idJornada, String codigoOrden): void	Coordina el retiro.
iniciarJornada(int idJornada): void	Valida, inicia visitas y registra cargos adicionales.
registrarResultado(int idJornada, String codigoOrden, ResultadoVisita resultado): void	Coordina el registro del resultado.
consultarOrdenes(): List<OrdenServicio>	Devuelve una copia de las órdenes.
consultarCuadrillas(): List<Cuadrilla>	Devuelve una copia de las cuadrillas.
generarReporteOrdenes(): String	Cuenta órdenes por categoría y estado.
generarReporteIngresos(): String	Totaliza ingresos por concepto y categoría.
generarReporteJornadas(): String	Muestra las jornadas sin cerrar, su cuadrilla, las horas asignadas y la capacidad disponible.

El método privado cargarDatosIniciales(): void se ejecuta una vez desde el constructor, sin generar ingresos y las cotizaciones utilizan objetos temporales para calcular costo y plazo; solo registrarOrden() guarda la orden y registra su cobro.

1.4.6. Aplicación

Metodo	Visibilidad	Proposito
`Aplicacion()`	Pública	Inicializa el sistema y el lector de entrada
`main(String[] args): void`	Pública y estática	Inicia el programa.
`ejecutar(): void`	Pública	Mantiene activo el menu.
`mostrarMenu(): void`	Privada	Presenta las opciones.
`procesarOpcion(int opcion): void`	Privada	Ejecuta la opcion seleccionada.
`leerEntero(String mensaje): int`	Privada	Lee un entero valido.
`leerDecimal(String mensaje): double`	Privada	Lee un decimal valido
`leerTexto(String mensaje): String`	Privada	Lee texto no vacio.

Si conservamos el ultimo codigo, tambien deben figurar `leerServicio(boolean registro): OrdenServicio`, `mostrarCotizacion(OrdenServicio orden): void` y `leerSiNo(String mensaje): boolean`, todos privados

1.4.7. Enumeraciones

Zona, EstadoOrden, EstadoJornada, ResultadoVisita y ConceptoIngreso restringen la información a las opciones válidas del sistema.

1.4.8.Constructores

Reciben los datos por parámetros y reutilizan la inicialización del padre mediante super(...). OrdenServicio inicializa los datos comunes; Instalacion valida los puntos; e InstalacionElectricaCertificada**,** los materiales. SistemaTecniHogar verifica que los códigos sean únicos.

1.5. Relación entre instalación e instalación eléctrica certificada

InstalacionElectricaCertificada hereda de Instalacion porque ambas comparten los puntos y la forma de calcular el costo de instalación. La certificada añade el cobro del certificado y la garantía, requiere dos días más y permite un máximo de dos visitas.

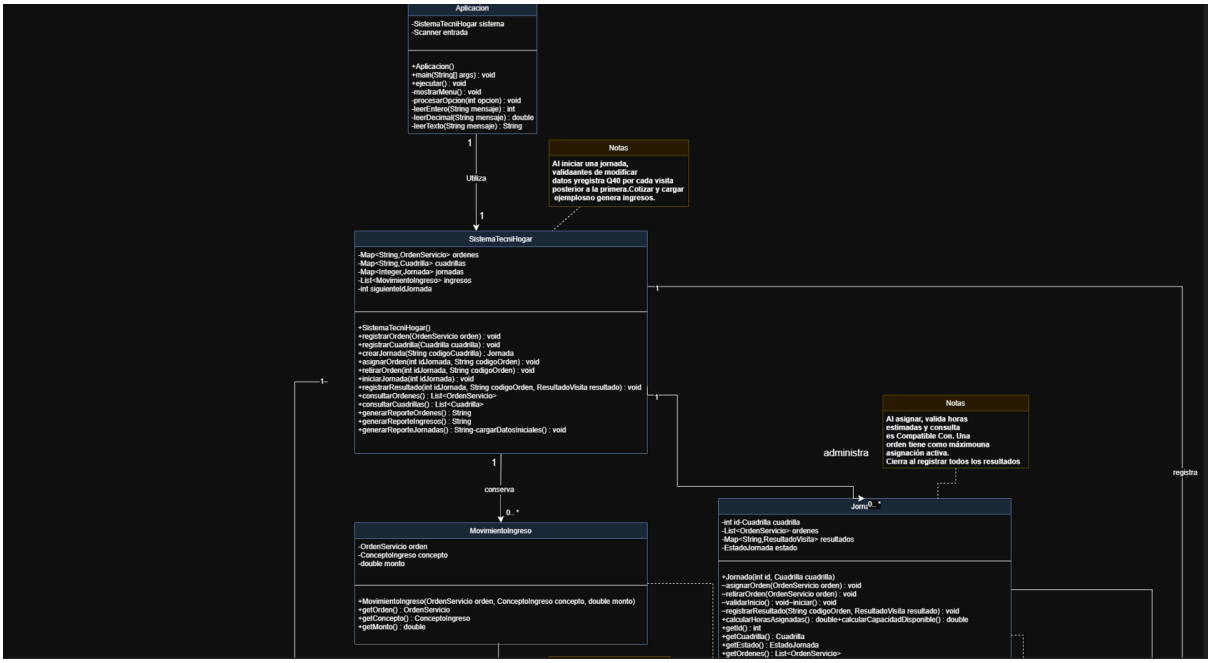
También podrían heredar directamente de OrdenServicio, pero habría que repetir o separar la lógica que comparten y por eso, heredar de Instalacion permite aprovechar lo que ya está definido.

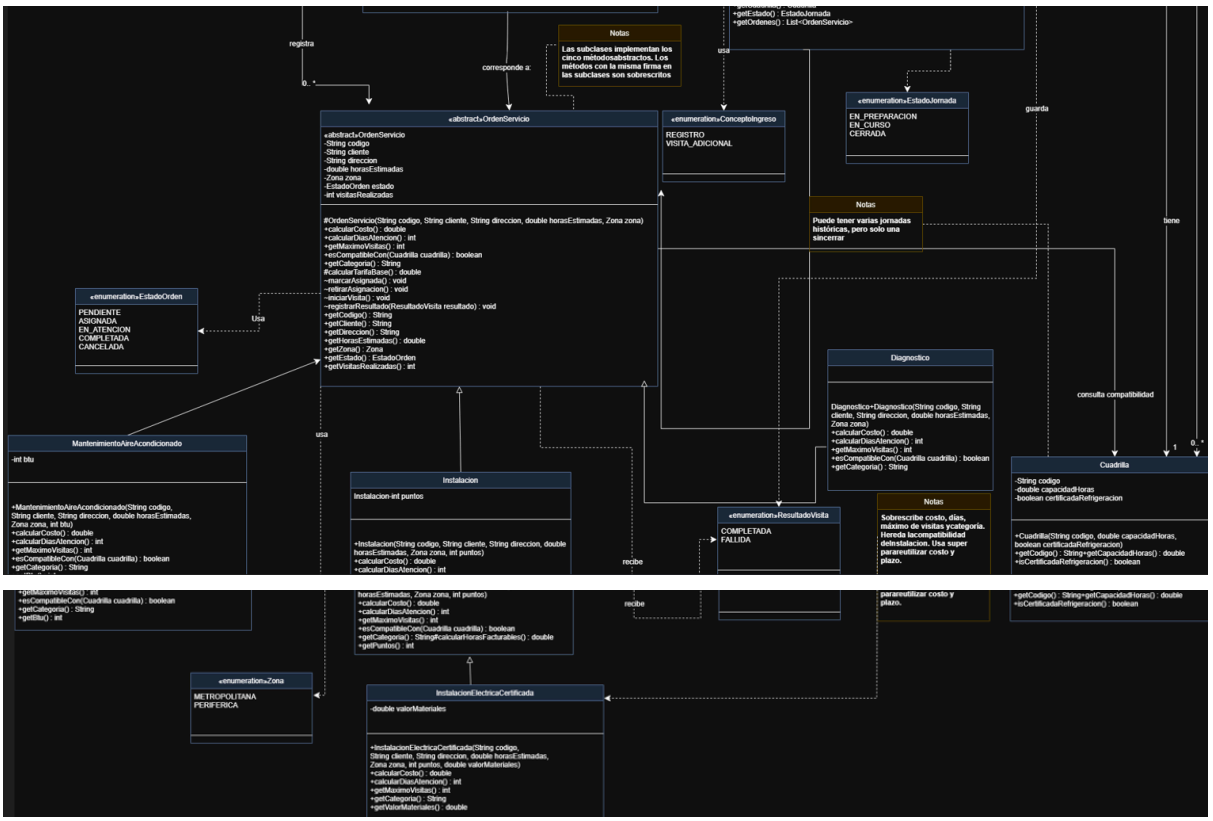
1.6. Incorporación de nuevas categorías

Para agregar otro servicio, se crea una clase que herede de OrdenServicio, o de Instalacion si comparte sus reglas. Allí se definen sus datos, validaciones, costo, tiempo de atención, requisitos de cuadrilla y límite de visitas. También se agrega la opción al menú.

Las jornadas pueden seguir funcionando porque consultan las reglas de cada servicio mediante sus métodos. Si el nuevo servicio requiere una certificación distinta, habría que agregar esa información a Cuadrilla. Los reportes tampoco necesitan cambios si agrupan las órdenes según la categoría que devuelve cada objeto.

2. Diseño UML





3. Enlace de GitHub

https://github.com/ntp90-cmd/Laboratorio-No.3---Herencia.git